

11785 Lec 24 GAN

Jun 12, 2026

Discriminative vs Generative Models

Given a distribution of inputs X and labels Y .

Discriminative models

- Discriminative models learn conditional distribution $P(Y | X)$
- Learns decision boundary between classes.
- Limited scope. Can only be used for classification tasks.
- E.g. Logistic regression, SVM etc.

Generative models

- Generative models learn the joint distribution $P(Y, X)$
- Learns actual probability distribution of data.
- Can do both generative and discriminative tasks.
- E.g. Naïve Bayes, Gaussian Mixture Model etc.
- Harder problem, requires a deeper understanding of the distribution than discriminative models.

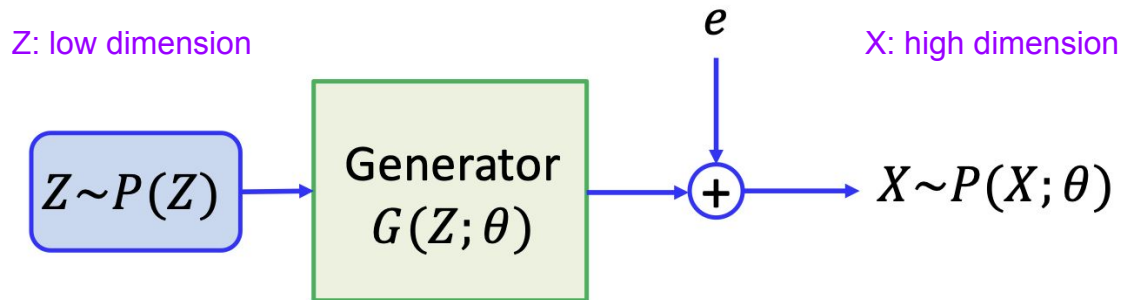
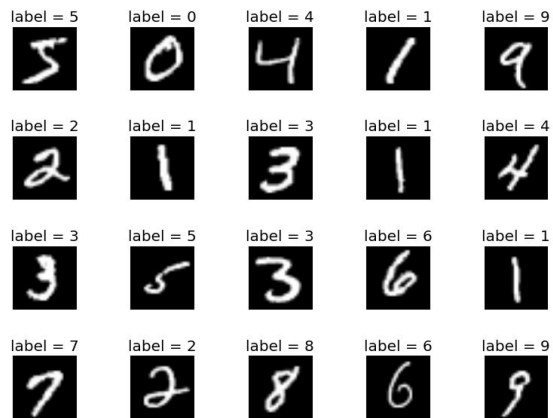
Explicit distribution models

- Calculates $P(x)$ for all x
probability of samples
- E.x. Gaussian Mixture Models; Naive Bayes; VAE

Implicit distribution models

- Generate $x \sim X$ (**draw samples**)
- E.x. GANs

VAE via Digit Classification



Latent variable

E.x. hidden factors:
 $Z = (z_1, z_2, z_3)$

- **thickness**
- **angle**
- **size**

Produces: μ

**average image
corresponding to
the characteristics**

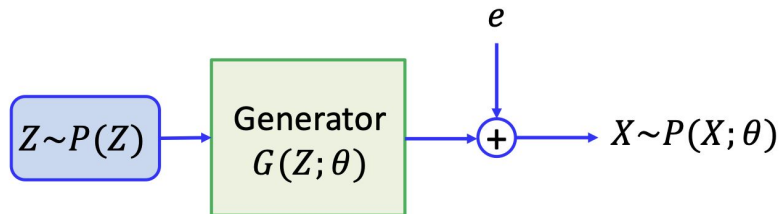
$X = \mu + e$

$X | \mu \sim N(\mu, \sigma^2 I)$

Conditional on Z , $X \sim \text{Gaussian}$
Training: X is real input
Generating: X is not real

Z captures hidden sources of
variation in the images

VAE to GAN



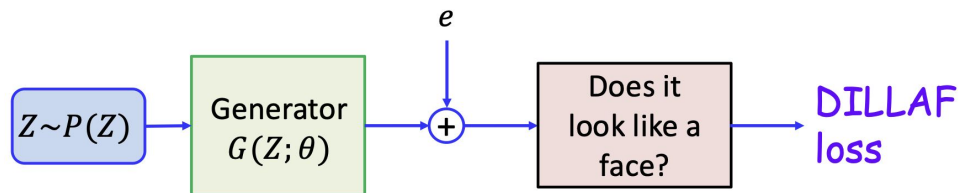
- Generator is a decoder of a VAE
- Trained by maximizing the *likelihood* of the data
 - Likelihood maximization does not actually relate to whether the output *actually* looks like a face

- Think of two clusters with different centers.
- Maximum likelihood is centered at their middle.
- The likelihood optimum exists, but the result is not representative of either cluster.

VAE: Which distribution makes the observed data most likely?

GAN: tell fake from real, and generate as real as possible

Replacing negative log likelihood with a more relevant loss



What are GANs

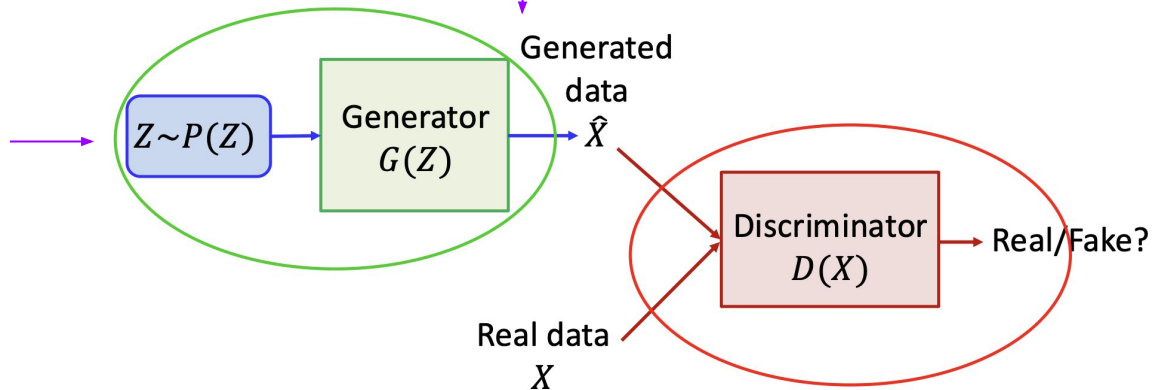


Generative Models which generate data similar to the training data .
E.g. Variational Autoencoders (VAE)

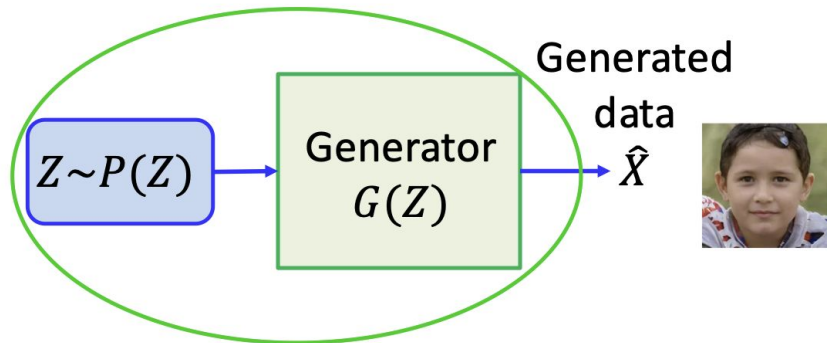
Adversarial Training
GANS are made up of two competing networks (adversaries) that are trying to beat each other.

- The generator learns a mapping from latent variables Z and directly generates $\hat{X}$.
- It does not first estimate an explicit distribution $p(X)$ and then sample from it.
- Hence **GAN is an implicit generative model**.

Generator input Z can be sampled from a known prior, e.g. standard Gaussian



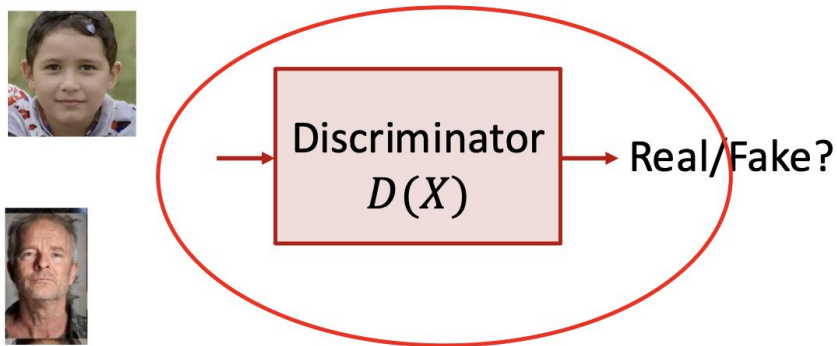
The Generator



- **The generator** produces realistic looking $X = G(z)$ from a latent vector Z
- Generator input Z can be sampled from a known prior, e.g. standard Gaussian
- **Goal:** generated distribution, $P_G(X)$ matches the true data distribution $P_X(X)$
 - $P_G(X)$ is the more “memorable” notation for $P_{\hat{X}}(X)$, the probability that a generated sample $\hat{X}$ takes the value X

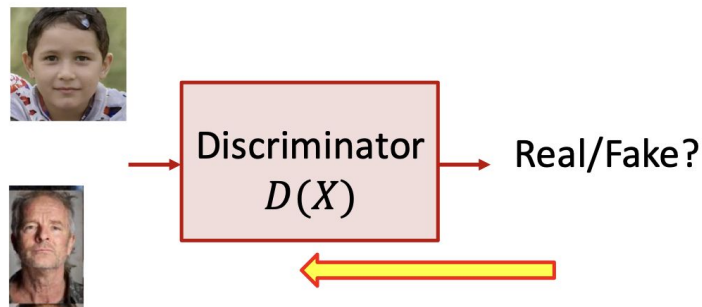
- P_G exists mathematically, but we **do not explicitly model or evaluate $P_G(x)$** .
- We directly generate samples $x=G(z)$, and compare generated samples with real samples using the discriminator.
- If the discriminator can no longer distinguish them, then P_G is close to P_X

The Discriminator



- Discriminator $D(X)$ is trained to tell the difference between real and generated (fake) data
 - Specifically, data produced by the generator
 - If a perfect discriminator is fooled, the generated data cannot be distinguished from real data

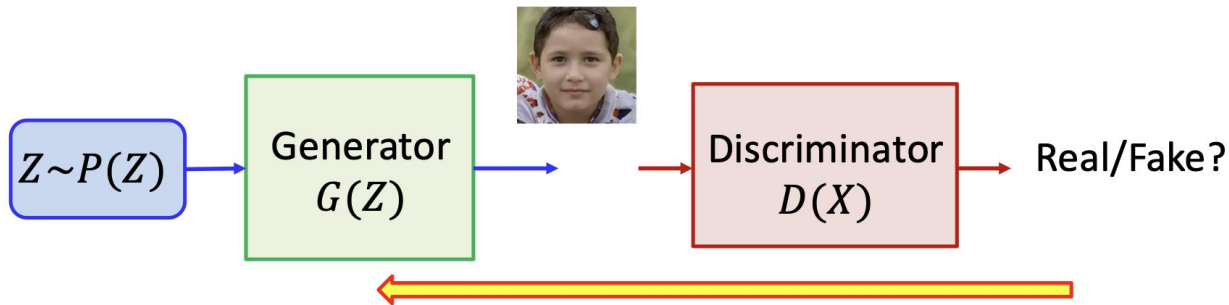
Training the discriminator



- **Training the discriminator:**

- The discriminator is provided training examples of real and synthetic faces
 - The discriminator is trained to minimize its classification loss
 - Minimize error between actual and predicted labels
 - Discriminator parameters are trained such that
 - $D(X) = 1$ for real faces
 - $D(X) = 0$ for synthetic faces (i.e $1 - D(X) = 1$)
- Maximize $\log(D(X))$ for real faces
 - Maximize $\log(1 - D(X))$ for synthetic faces

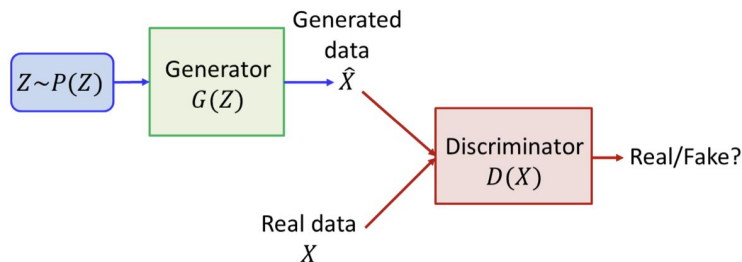
Training the generator



- **Training the generator:**

- The discriminator's loss is backpropagated to the generator
- The generator is trained to *maximize* the discriminator loss
 - It is trained to “fool” the discriminator
- Generator parameters are trained such that
 - $D(G(Z)) = 1$ (i.e. $1 - D(G(Z)) = 0$)
 - Minimize $\log(1 - D(G(Z)))$

The GAN formulation

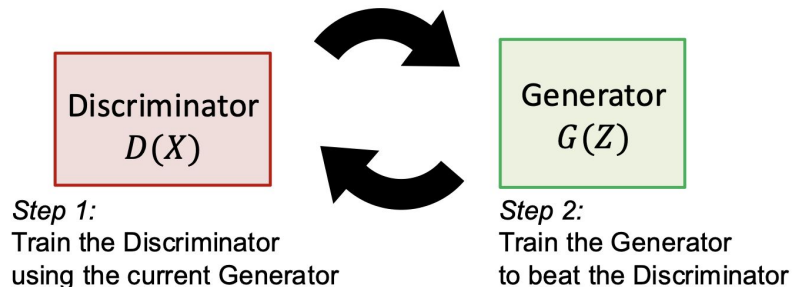


- The original GAN formulation is the following min-max optimization

$$\min_G \max_D E_X \log D(X) + E_Z \log(1 - D(G(Z)))$$

- Objective of D : $D(X) = 1$ and $D(G(Z)) = 0$
- Objective of G : $D(G(Z)) = 1$

How to Train a GAN?

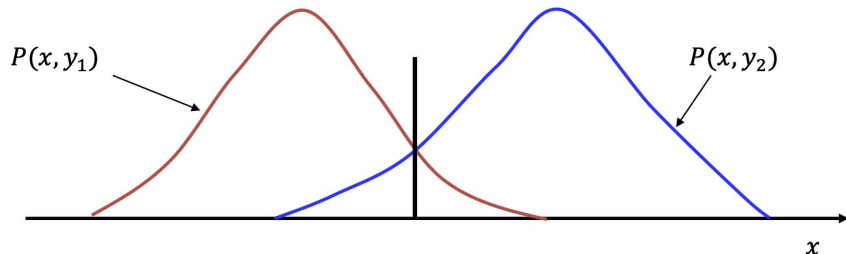


The discriminator is not needed after convergence

- max D** • Discriminator:
 - For real data X , Maximize $\log(D(X))$
 - For synthetic data Maximize $\log(1 - D(\hat{X}))$
- min G** • Generator
 - Minimize $\log(1 - D(\hat{X}))$

The perfect discriminator:

Consider a binary classification problem



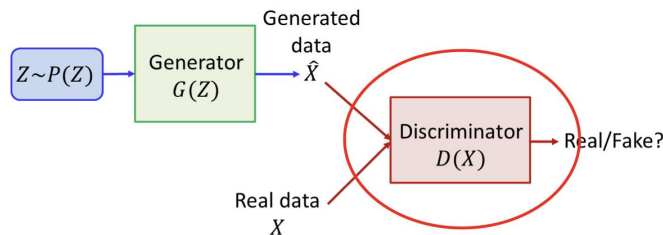
- The a posteriori probability of the classes for any instance $x = X$ is

$$P(y_i|X) = \frac{P(X, y_i)}{P(X, y_1) + P(X, y_2)}$$

- The perfect decision boundary is where $P(y_1|X) = P(y_2|X)$
 - The perfect discriminator will compute $P(y_i|X)$ for each class
 - It will assign any X to the class with the higher $P(y_i|X)$

41

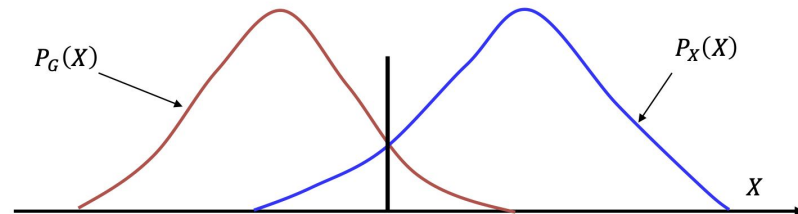
The optimal discriminator



- The **optimal discriminator** would be a Bayesian classifier

$$D(X) = \frac{P_X(X)}{P_X(X) + P_G(X)}$$

- Assuming uniform prior



42

$Y = \begin{cases} 1 & \text{real } (X \sim P_X) \\ 0 & \text{generated } (X \sim P_G) \end{cases}$

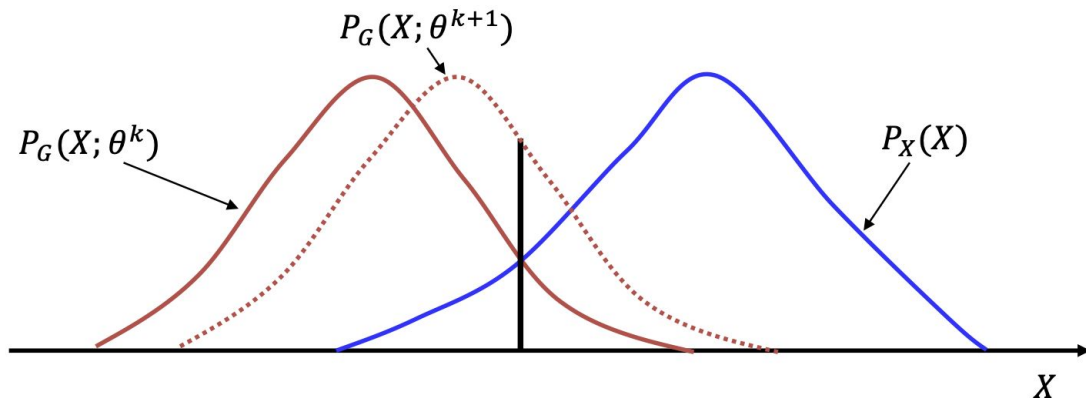
Discriminator: $D(x) = P(Y=1|x) = \frac{P(X|Y=1) \cdot P(Y=1)}{P(X)}$ where $P(X|Y=1) = P_X(x)$ and $P(X|Y=0) = P_G(x)$

uniform prior $\rightarrow P(Y=1) = P(Y=0) = \frac{1}{2}$
 $P(X) = \frac{1}{2} P_X(x) + \frac{1}{2} P_G(x)$

$D(x) = \frac{\frac{1}{2} P_X(x) \cdot \frac{1}{2}}{\frac{1}{2} P_X(x) + \frac{1}{2} P_G(x)} = \frac{P_X(x)}{P_X(x) + P_G(x)}$

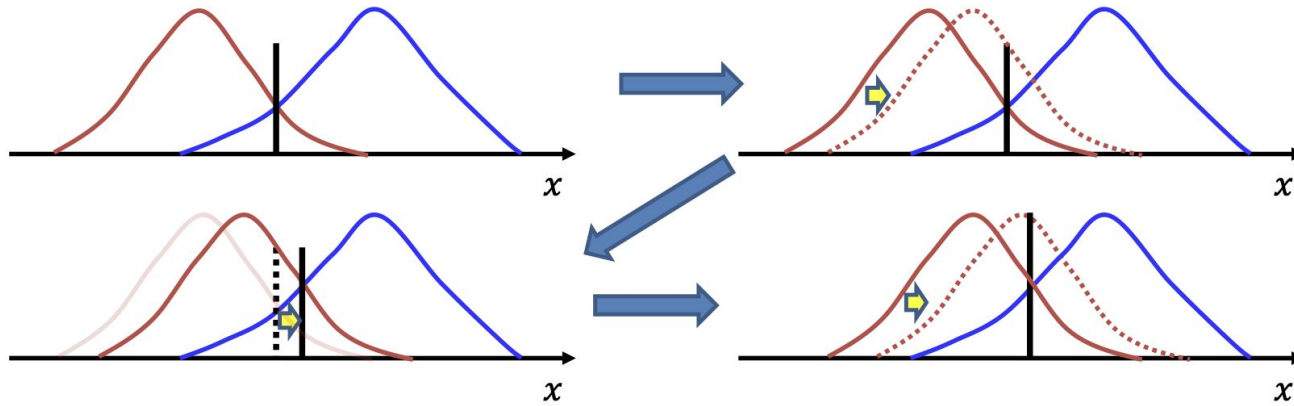
Updating the Generator:

Fooling the perfect discriminator

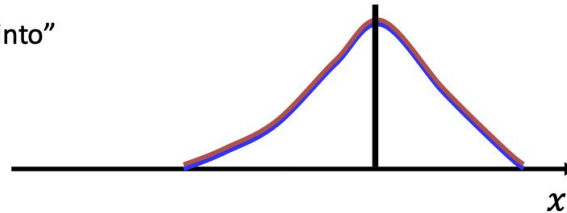


- Relearn generator parameters so that the new distribution of generated data “fools” the discriminator
 - By moving it into the region assigned to the other class by the (perfect) discriminator

The iterated learning

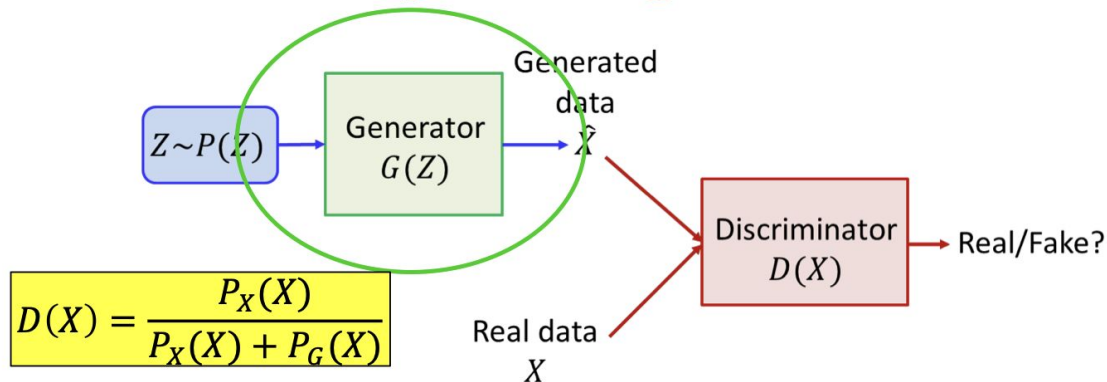


- Discriminator learns perfect boundary
- Generator moves its distribution past the boundary “into” the real distribution
- Discriminator relearns new “perfect” boundary
- Generator shifts distribution past new boundary
- ...
- In the limit Generator’s distribution sits perfectly on “real” distribution and the perfect discriminator is still random



Analysis of optimal behavior:

The optimal generator



$$\min_G \max_D E_X \log D(X) + E_Z \log(1 - D(G(Z)))$$

- With a perfect discriminator:

$$\begin{aligned} L &= E_{X \sim P_X(X)} \log D(X) + E_{X \sim P_G(X)} \log(1 - D(X)) \\ &= E_{X \sim P_X(X)} \log \left(\frac{P_X(X)}{P_X(X) + P_G(X)} \right) + E_{X \sim P_G(X)} \log \left(\frac{P_G(X)}{P_X(X) + P_G(X)} \right) \end{aligned}$$

What statistical quantity is being minimized between P_G and P_X ?

The KL Divergence ✗

$$KL(P, Q) = \sum_X P(X) \log(P(X)/Q(X))$$

- What are the problems with this?

$$KL(Q, P) = \sum_X Q(X) \log(Q(X)/P(X))$$

- What are the problems with this?

KL is not symmetric, and runs into issues if either P or Q become 0 (whichever is entirely inside the log)

The Jensen Shannon Divergence ✓

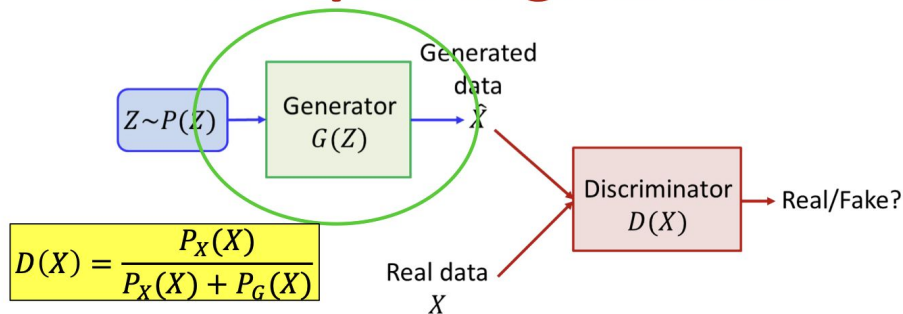
$$\begin{aligned} JSD(P, Q) \\ = 0.5 KL(P, 0.5(P + Q)) + 0.5 KL(Q, 0.5(P + Q)) \end{aligned}$$

- If the term inside the log is 0, both P and Q are 0
 - $0 \log 0 = 0$, so there are no problems
- Also, this is symmetric:

$$JSD(P, Q) = JSD(Q, P)$$

Analysis of optimal behavior:

The optimal generator



$$\min_G \max_D E_X \log D(X) + E_Z \log(1 - D(G(Z)))$$

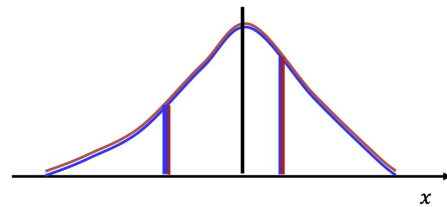
- **With a perfect discriminator:**

$$\begin{aligned} L &= E_{X \sim P_X(X)} \log D(X) + E_{X \sim P_G(X)} \log(1 - D(X)) \\ &= E_{X \sim P_X(X)} \log \left(\frac{P_X(X)}{P_X(X) + P_G(X)} \right) + E_{X \sim P_G(X)} \log \left(\frac{P_G(X)}{P_X(X) + P_G(X)} \right) \end{aligned}$$

- This is just the Jensen-Shannon divergence between $P_X(X)$ and $P_G(X)$ to within a scaling factor and a constant

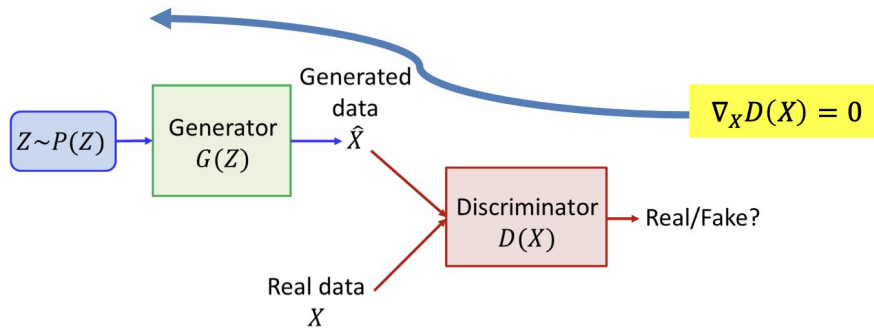
$$L = 2JSD(P_X(X), P_D(X)) - \log 4$$

The optimal generator with the optimal discriminator



- The generator of the fully optimized GAN will generate $P_G(X) = P_X(X)$, i.e. the distribution of the generated data will be identical to that of the original data
- At any X , $P_G(X) = P_X(X)$
 - i.e. $D(X) = \frac{P_X(X)}{P_X(X) + P_G(X)} = 0.5$
 - The derivative of $D(X)$ w.r.t $X = 0$

The optimal generator with the optimal discriminator



- $\nabla_X D(X) = 0$
- All derivatives going backward are 0
- There will be no further updates

- Generator receives zero gradient
- Updates stop because the generated distribution matches the real distribution

Min-Max Stationary Point

- There exists a stationary point:
 - If the generated data exactly matches the real data, the discriminator outputs 0.5 for all inputs
 - If discriminator outputs 0.5, the gradients for the generator is flat, so generator does not learn
 - Unfortunately, this is also true of a random discriminator
- Stationary points need not be stable (depends on the exact GANs formulation and other factors)
 - Generator may overshoot some values or oscillate around the optimum
 - A discriminator with unlimited capacity can still assign an arbitrarily large distance to 2 similar distributions

- Having gradient = 0 is not enough to guarantee we reached the desired solution.
- Possible a bad discriminator also gives 0.5
- Stationary points in GANs are not automatically good solutions.

VAEs vs GANs

$$\theta^* = \arg \min_{\theta} -\log P(X; \theta)$$

VAEs

- Minimizing the KL divergence between distributions of synthetic and true data
- Uses an encoder to predict latent distributions to optimize generator
- More complex formulation
- Simpler optimization. Trains faster and more reliably
- Results are blurry

Try to cover the full data distribution

Good VAE: cat, dog
Weak VAE: uncertain of cat or dog,
blurry combination

$$\text{Optimize: } \min_G \max_D E_X \log D(X) + E_Z \log(1 - D(G(Z)))$$

GANs

- Minimizing the Jensen-Shannon divergence between distributions of synthetic and true data
- Use a discriminator to optimize generator
- Simpler formulation
- Noisy and difficult optimization
- Sharper results

- Not a standard optimization
- Not as stable

Blurry output likely regarded as fake, so tend towards sharp

Try to get realistic individual samples⁶⁹

Good GAN: cat, dog
Weak GAN: only cat (lose info)

E.x. Cat,
Dog

Problems of Vanilla GANs

1. Vanishing gradients:

Generator receives no meaningful gradient for learning

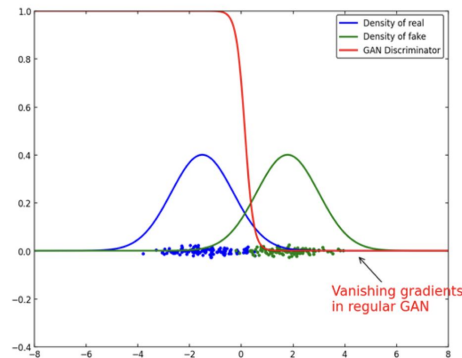
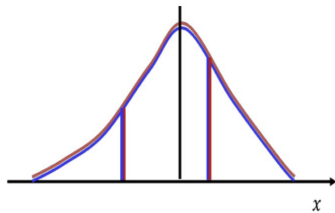
Causes:

i. Discriminator issue

- too strong $\rightarrow$ outputs $\approx 0 / 1$
- too weak $\rightarrow$ unreliable feedback

- A weak discriminator cannot distinguish real from fake, outputs 0.5 everywhere, and thus provides no useful gradients

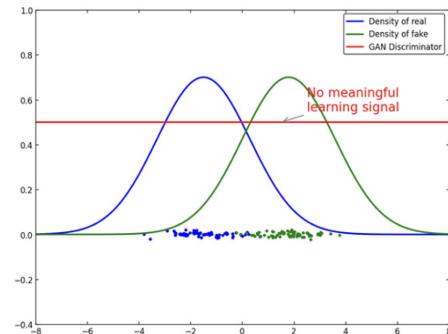
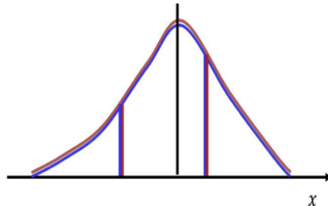
The problem with a strong discriminator



- A strong discriminator perfectly separates real and fake, and does not pass useful gradients to guide the generator.

79

The problem with a weak discriminator



Problems of Vanilla GANs

1. Vanishing gradients:

Generator receives no meaningful gradient for learning

Causes:

i. Discriminator issue

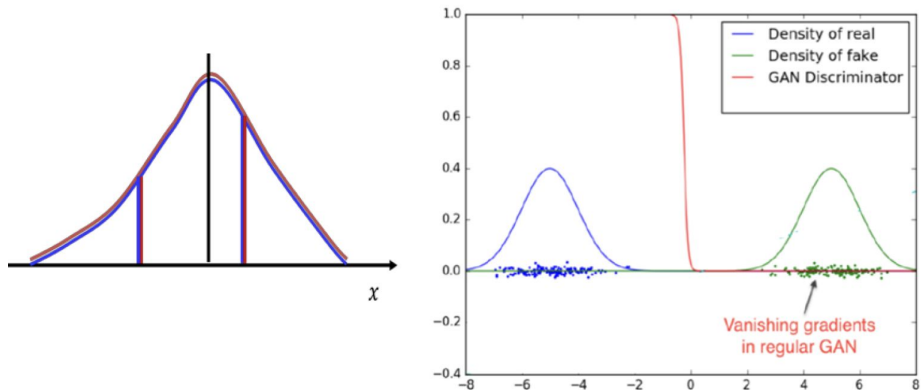
- too strong $\rightarrow$ outputs $\approx 0 / 1$
- too weak $\rightarrow$ unreliable feedback

ii. Loss issue

- **non-overlapping distributions**
- **sigmoid saturation**

Strong discriminator $\rightarrow$ saturated sigmoid

Problem with sigmoid loss



- Sigmoid loss leads to saturation, resulting in near-zero gradients

Problems of Vanilla GANs

1. Vanishing gradients:

Generator receives no meaningful gradient for learning

Causes:

i. Discriminator issue

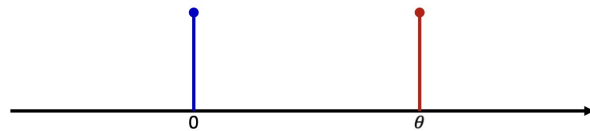
- too strong $\rightarrow$ outputs $\approx 0 / 1$
- too weak $\rightarrow$ unreliable feedback

ii. Loss issue

- non-overlapping distributions
- sigmoid saturation

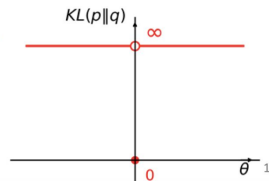
iii. Divergence issue

Problem with KL divergence

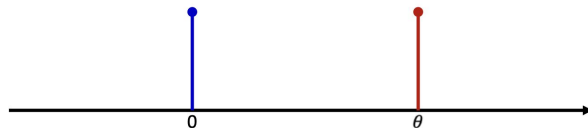


$$KL(p||q) = \int_x p(x) \log \frac{p(x)}{q(x)}$$

- Real data is a point mass at 0
- Generated data is a point mass at θ
- If $\theta \neq 0$, $p(0) \log \frac{p(0)}{q(0)} = 1 \log \frac{1}{0} = \infty$
- If $\theta = 0$, $1 \log \frac{1}{1} = 0$
- Not differentiable w.r.t. θ



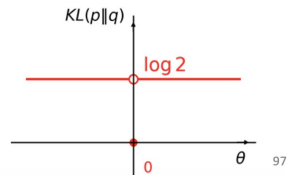
Problem with JS divergence



Calculate the average distribution and calculate the average KL to the average.

$$m(x) = \frac{p(x) + q(x)}{2}$$
$$JS(p||q)$$

- Real data is a point mass at 0
- Generated data is a point mass at θ
- If $\theta \neq 0$, $\frac{1}{2} [1 \log \frac{1}{0.5} + 1 \log \frac{1}{0.5}] = \log 2$
- If $\theta = 0$, $\frac{1}{2} [1 \log \frac{1}{1} + 1 \log \frac{1}{1}] = 0$
- Not differentiable w.r.t. θ



Detailed calculations

Problem with KL divergence



$$KL(p\|q) = \int_x p(x) \log \frac{p(x)}{q(x)} \quad p(x) = \begin{cases} 1 & x=0 \\ 0 & \text{else} \end{cases} \quad q(x) = \begin{cases} 1 & x=\theta \\ 0 & \text{else} \end{cases}$$

Case 1: $\theta \neq 0$

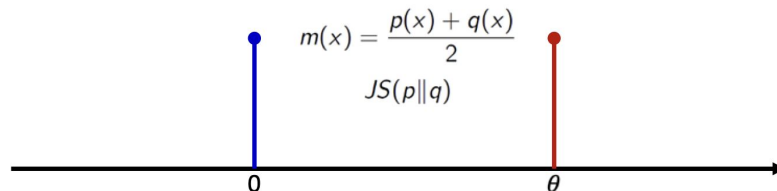
$$KL(p\|q) = p(0) \log \frac{p(0)}{q(0)} + p(\theta) \log \frac{p(\theta)}{q(\theta)} = 1 \log \frac{1}{0} + 0 \log \frac{0}{1} = \infty$$

Case 2: $\theta = 0$

$$KL(p\|q) = p(0) \log \frac{p(0)}{q(0)} = 1 \log \frac{1}{1} = 0$$

89

Problem with JS divergence



Case 1: $\theta \neq 0$

$$M = \frac{p+q}{2}, \quad M(0) = 0.5, \quad M(\theta) = 0.5$$

$$JS(p\|q) = \frac{1}{2} KL(p\|M) + \frac{1}{2} KL(q\|M) = \frac{1}{2} \left(1 \log \frac{1}{0.5} + 0 \log \frac{0}{0.5} \right) + \frac{1}{2} \left(0 \log \frac{0}{0.5} + 1 \log \frac{1}{0.5} \right) = \log 2$$

Case 2: $\theta = 0$

$$M = p = q, \quad M(0) = 1$$

$$JS(p\|q) = \frac{1}{2} KL(p\|M) + \frac{1}{2} KL(q\|M) = \frac{1}{2} \left(1 \log \frac{1}{1} \right) + \frac{1}{2} \left(1 \log \frac{1}{1} \right) = 0$$

95

How to solve the JS divergence issue?

Least-Squares GAN

- Use an L_2 loss instead of cross-entropy
- Generator tries to minimize L_2 loss

$$E_{Z \sim P(Z)} \left[\left(D(G(Z)) - 1 \right)^2 \right]$$

- Discriminator tries to minimize L_2 loss

$$E_{X \sim P(X)} \left[\left(D(X) - 1 \right)^2 \right] + E_{Z \sim P(Z)} \left[\left(D(G(Z)) \right)^2 \right]$$

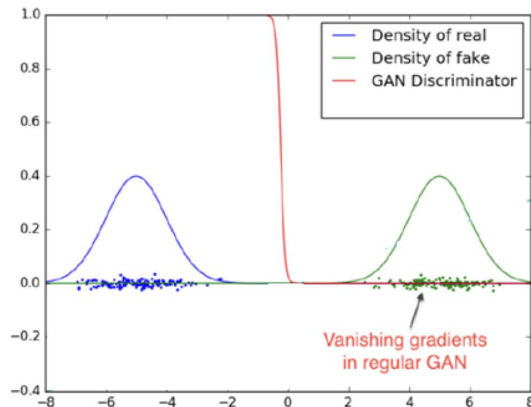
- Less squashing and fewer large values than cross-entropy

Optimize: $\min_G \max_D E_X \log D(X) + E_Z \log(1 - D(G(Z)))$

Generator wants fake to look real: $D(G(Z)) \rightarrow 1$

Discriminator:
real: $D(X) \rightarrow 1$
fake: $D(G(Z)) \rightarrow 0$

Least-squares GAN



LSGAN reduces the saturation speed, but it does not completely remove all vanishing-gradient problems.

- The loss is better behaved than KL
- But the discriminator itself can continue to be misbehaved
- Can we do better?

Wasserstein Distance

The $\text{total } \sum \text{mass} \times \text{distance}$ required to transform one distribution to another.

- Intuitive, symmetric measure of divergence
- Hard to calculate because requires solving “optimal transport”
- You have some distribution of products at some warehouses and you need to make some other distribution of products at those warehouses
- Move the products to the target distribution minimizing $\sum \text{mass} \times \text{distance}$
- Creating the optimal “transport plan” can be NP-hard



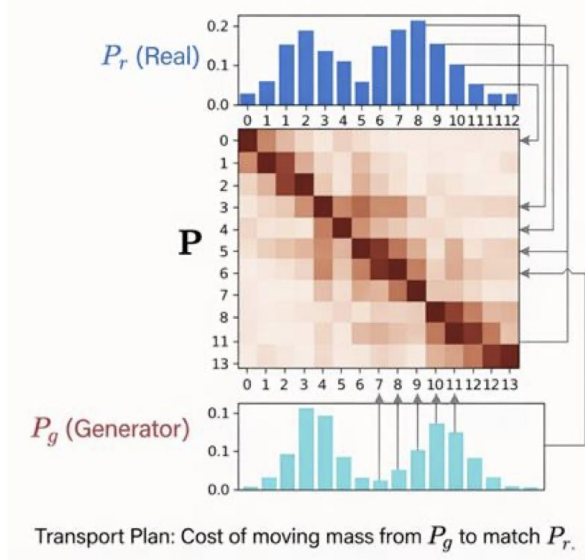
- Vanilla GAN and LSGAN can only tell two distributions are different
- WGAN can tell how different

W distance provides useful (different) gradients to guide the generated distribution toward the real distribution

Optimize by choosing the way of minimum total cost

Wasserstein Distance

Visual Intuition



Definition & Logic

The **Wasserstein Distance (W)** measures the minimum cost to transport probability mass from one distribution to another.

$$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$

- Heatmap: amount of mass transported
- Since transport cost = mass $\times$ distance, the optimal plan favors nearby moves, leading to larger mass assignments near the diagonal.

- Real data is a point mass at 0
- Generated data is a point mass at θ



Move generated to real:

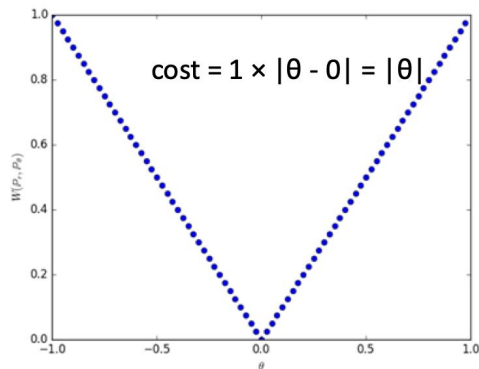
- cost = mass $\times$ distance
= $1 \times \text{distance} = \text{distance} = |\theta|$

Hard to Calculate: Involves optimal transport, which is computationally intensive.

Wasserstein GANs: Two-mass case

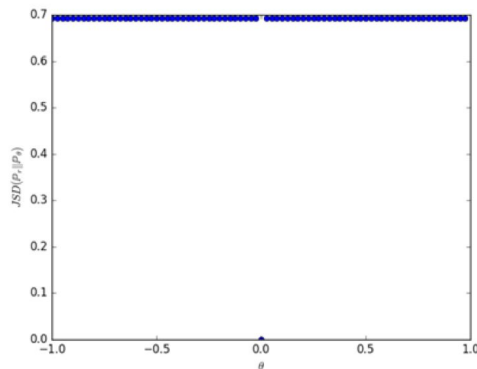
- Uses **Earth-mover (Wasserstein-1) distance** to measure the distance between two distributions

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|]$$



Wasserstein-1

Differentiable w.r.t θ Continuous and useful gradient everywhere



JS

Not Continuous

Wasserstein GAN

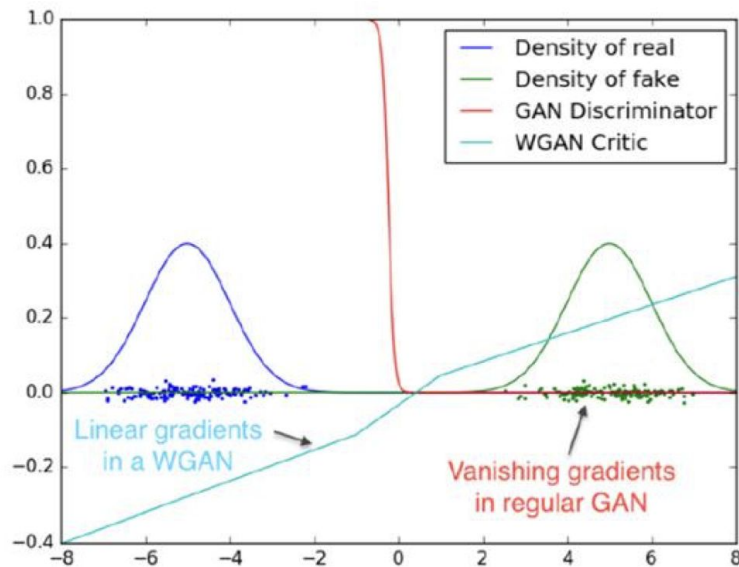


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

Wasserstein GAN vs regular GAN which is which?



Optimal transport for W distance is hard, hence:

Kantorovich-Rubinstein Duality

Since the infimum in the primal form is intractable to compute directly, we use the dual form.

$$W(P_r, P_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim P_r} [f(x)] - \mathbb{E}_{x \sim P_g} [f(x)]$$

as different as possible

The Critic. Replaces the Discriminator. Outputs a raw score (realness), not a probability.

Maximization. We train the critic to maximize the difference between scores for real and fake data.

1-Lipschitz Constraint. The function f cannot change too quickly. The gradient norm must be ≤ 1 everywhere.

Without the Lipschitz constraint, the critic would assign $-\infty$ to fake samples and $+\infty$ to real samples, causing the loss to explode.

$$W(P_r, P_g) = \inf_{\gamma} E_{(x,y) \sim \gamma} [\|x - y\|]$$

- Need to find optimal transport plan γ
- Too many possible transports

Solution: turn optimal transport into a neural network

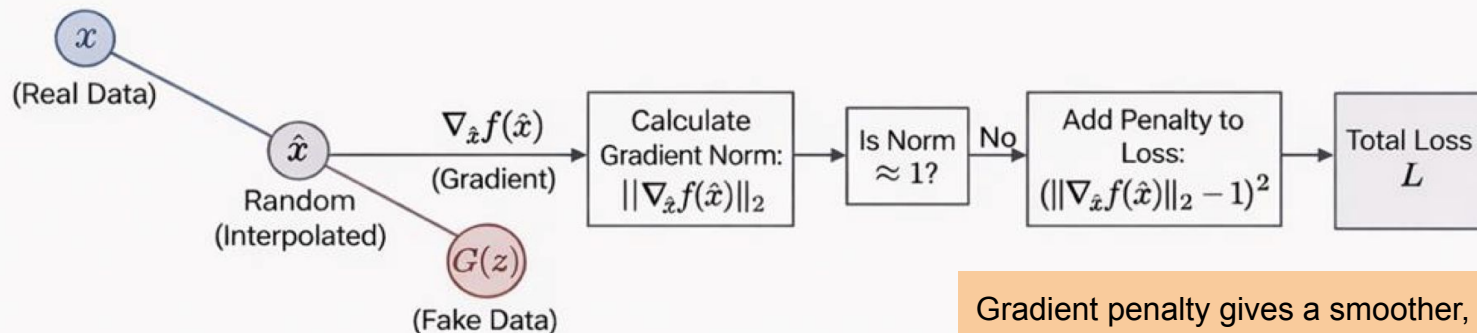
- Optimize over functions f

Instead of explicitly finding how probability mass should be transported between two distributions, we find a function that maximizes the difference between its expected values under the real and generated distributions.

WGAN with Gradient Penalty (WGAN-GP)

To enforce the Lipschitz constraint, we avoid weight clipping (which leads to optimization issues) and instead penalize the gradient norm directly.

$$L = \underbrace{\mathbb{E}[f(G(z))] - \mathbb{E}[f(x)]}_{\text{Original Wasserstein Estimate}} + \underbrace{\lambda \mathbb{E}_{\hat{x}} [(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2]}_{\text{Gradient Penalty}}$$



Gradient penalty gives a smoother, more expressive critic and more stable training than WGAN.

Real life applications of GAN

Creating Data for Self-Driving Cars

- Companies like Waymo a part of Google's parent company Alphabet use GANs to create synthetic data for training self-driving cars. These cars need a lot of data to learn how to drive safely in different situations such as varying road conditions, weather and environments.

DeepMind's GAN Applications in Healthcare

- DeepMind a research company owned by Google uses GANs to help in healthcare especially with medical images. They use GANs to turn low-quality MRI scans into high-quality images which helps doctors make better diagnoses without needing expensive equipment.

Disadvantages

- **Training Difficulty:** GANs are complex and computationally expensive, requiring significant resources for effective training.
- **Overfitting:** It can overfit to training data and produce synthetic data that lacks diversity and fails to generalize well.
- **Bias and Fairness:** It can learn biases present in the training data which can lead to discriminatory outputs.
- **Interpretability:** GANs are often referred to as black-box models making it difficult to understand how they arrive at their results.
- **Quality Control:** They may generate unrealistic or irrelevant data if not trained properly affecting the quality of the synthetic data.